Model Predictive Control of an Inverted Pendulum

OBJECTIVE.

Design a model predictive controller (MPC) for the inverted pendulum with a bounded actuator force and compare its nonlinear response with the linear quadratic regulator (LQR) from Mini Project 3.

METHODOLOGY.

The nonlinear Simulink plant from Mini Project 3 is retained with cart mass 2 kg, pendulum mass 0.5 kg, rod length 1 m, pivot damping 0.01 N m s/rad, and gravity 9.81 m/s². The state vector $z = \left(x, \dot{x}, \theta, \dot{\theta}\right)^T$ contains cart position, cart velocity, pendulum angle from upright, and angular velocity. The upright linearization $\dot{z} = Az + Bu$ is the MPC prediction model. The simulation supplies the four plant states directly to the controller.

The LQR uses continuous state feedback. The Simulink MPC Controller block updates every 0.05 s. Its prediction horizon is 50 steps (2.5 s), and its control horizon is 10 steps. The optimizer penalizes state deviations, force, and force changes. The output weights are $\left(2, \sqrt{2}, \sqrt{300}, \sqrt{10}\right)$; the force and force-rate weights are 0.707 and 0.1, respectively. Both controllers use an applied force limit of ± 10 N. MPC includes this limit in its optimization, whereas the LQR command is clipped by the actuator.

Both cases start at rest with $\theta = 5°$, zero cart displacement, and zero disturbance. Each nonlinear simulation lasts 30 s. Settling requires $|\theta| \leq 1°$ and $|x| \leq 0.05$ m thereafter. Force effort is $E_u = \int_0^{30} u(t)^2 \, \mathrm{d}t$; MPC effort uses the held input intervals. Appendix A provides the controller construction and comparison code.

RESULTS AND DISCUSSION.

Figure 1 compares the responses. The LQR and MPC joint settling times are 4.24 and 4.36 s, respectively. Neither controller reaches the ± 10 N limit in this test.

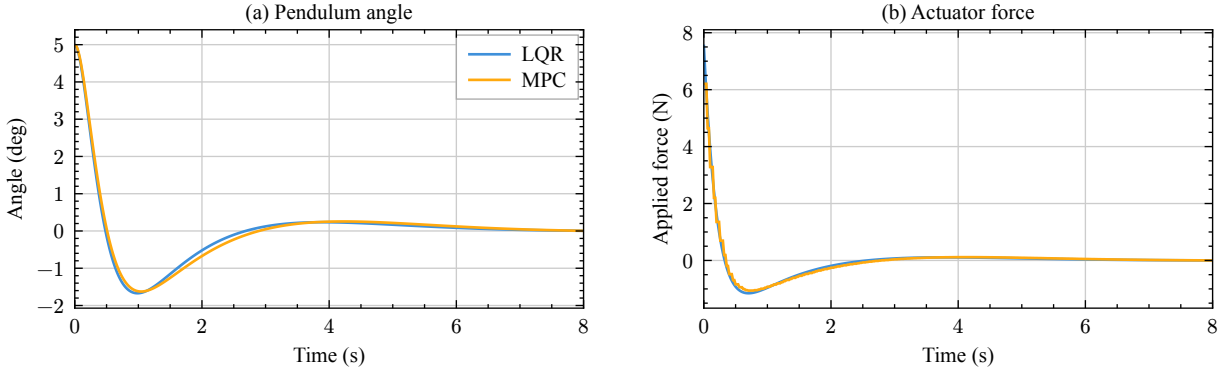


Figure 1: Matched nonlinear response from a 5° release. The first 8 s are shown.

Measure	LQR	MPC
Joint settling (s)	4.24	4.36
Peak cart travel (m)	0.3	0.32
Peak force (N)	7.58	6.2
Force effort (N² s)	5.14	4.85

Table 1: Comparison over the full 30 s simulation.

The 2.5 s prediction horizon was selected from three tests with the same weights. A 1 s horizon did not meet the recovery criterion within 30 s; a 1.75 s horizon settled in 9.4 s. The selected horizon settled in 4.36 s and reduced peak cart travel. The ± 10 N limit matches Project 3. MPC reduces force effort while increasing peak cart travel relative to LQR. These differences reflect the respective weights and update rates; the force constraint remains inactive.

Appendix A. MATLAB and Simulink implementation.

The Simulink model uses the nonlinear plant and state ordering from Mini Project 3. The following code constructs the MPC object, inserts its controller block into the copied model, and runs the matched comparisons.

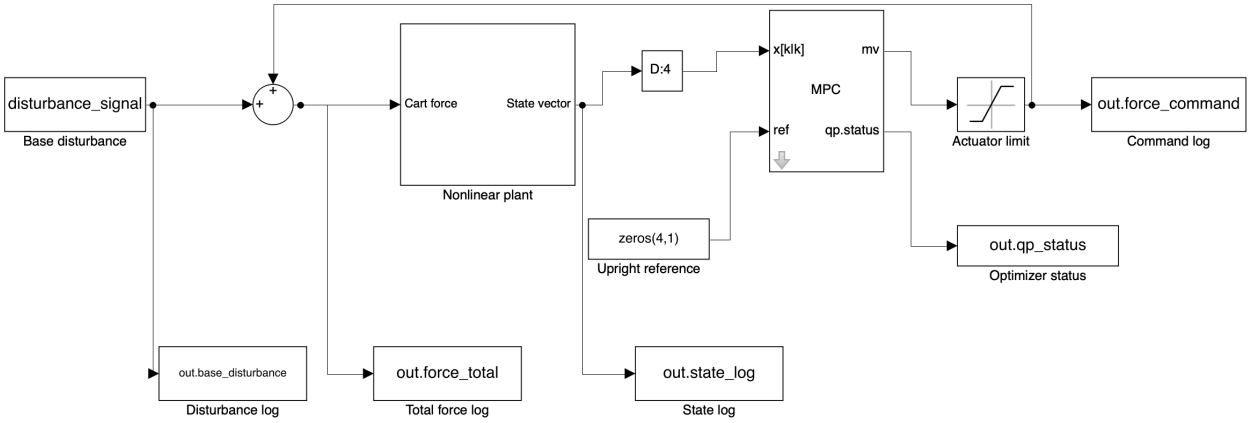


Figure 2: Simulink MPC implementation. The disturbance is zero for the reported tests.

```
function build_project4()
% Build and verify constrained MPC against the unaltered Project 3 LQR.
folder=fileparts(mfilename('fullpath')); cd(folder);
Simulink.fileGenControl('set','CacheFolder',fullfile(tempdir,'emch792-project4-cache'), ...
    'CodeGenFolder',fullfile(tempdir,'emch792-project4-code'),'createDir',true);
prior=fullfile(folder,'..','mini_project_3');
[K,d]=design_lqr_controller(2,0.5,1,0.01,9.81);
Ts=0.05; p=50; m=10;
plant=ss(d.A,d.B,eye(4),zeros(4,1));
controller=mpc(plant,Ts,p,m);
controller.MV.Min=-10; controller.MV.Max=10;
controller.Weights.OutputVariables=[2 sqrt(2) sqrt(300) sqrt(10)];
controller.Weights.ManipulatedVariables=sqrt(0.5);
controller.Weights.ManipulatedVariablesRate=0.1;
% All four physical states are directly available from the simulated plant.
% Remove default unmeasured output disturbances from the augmented state.
setoutdist(controller,'model',ss(zeros(4,1)));
setEstimator(controller,'custom');
source=fullfile(prior,'inverted_pendulum_lqr.slx');
load_system(source); old='inverted_pendulum_lqr'; model='inverted_pendulum_mpc';
save_system(old,fullfile(folder,[model '.slx'])); close_system(old,0); load_system(model);
ws=get_param(model,'ModelWorkspace'); assignin(ws,'mpc_controller',controller);
delete_line(model,'Nonlinear plant/1','State feedback/1');
delete_line(model,'State feedback/1','Actuator limit/1');
delete_block([model '/State feedback']);
load_system('mpclib');
add_block('mpclib/MPC Controller',[model '/MPC Controller'], ...
    'Position',[625 215 695 305],'mpcobj','mpc_controller', ...
    'state_inport','on','return_qpstatus','on','md_inport','off');
add_block('simulink/Sources/Constant',[model '/Upright reference'], ...
    'Value','zeros(4,1)','Position',[570 160 650 190]);
add_block('simulink/Signal Attributes/Signal Specification',[model '/State dimensions'], ...
    'Dimensions','4','Position',[580 220 610 250]);
add_line(model,'Nonlinear plant/1','State dimensions/1','autorouting','on');
add_line(model,'State dimensions/1','MPC Controller/1','autorouting','on');
add_line(model,'Upright reference/1','MPC Controller/2','autorouting','on');
add_line(model,'MPC Controller/1','Actuator limit/1','autorouting','on');
add_block('simulink/Sinks/To Workspace',[model '/Optimizer status'], ...
    'VariableName','qp_status','SaveFormat','Timeseries', ...
    'Position',[740 315 860 345]);
add_line(model,'MPC Controller/2','Optimizer status/1','autorouting','on');
format_project4_layout(model);
save_system(model);
% Simulate unchanged Project 3 first; no saved state in the original is altered.
load_system(source);
```

```
lqrout=sim(old); mpcout=sim(model);
results.settings=struct('Ts_s',Ts,'prediction_horizon_steps',p, ...
    'prediction_horizon_s',Ts*p,'control_horizon_steps',m, ...
    'force_limit_N',10,'initial_angle_deg',5,'stop_time_s',30);
results.design=struct('A',d.A,'B',d.B,'lqr_K',K, ...
    'output_weights',controller.Weights.OutputVariables, ...
    'force_weight',controller.Weights.ManipulatedVariables, ...
    'force_rate_weight',controller.Weights.ManipulatedVariablesRate);
results.comparison.lqr=extract_case(lqrout,'LQR');
results.comparison.mpc=extract_case(mpcout,'MPC');
reference=jsondecode(fileread(fullfile(prior,'project3_results.json')));
results.validation.raw_peak_force_N=max(abs(mpcout.force_command.Data(:)));
assert(results.validation.raw_peak_force_N<=10+1e-7);
assert(all(mpcout.qp_status.Data(:)>0));
results.validation.qp_status_min=min(mpcout.qp_status.Data);
results.validation.qp_status_max=max(mpcout.qp_status.Data);
results.validation.lqr_angle_max_difference_deg=max(abs(results.comparison.lqr.theta_deg(:)-
reference.comparisons(1).lqr.theta_deg(:)));
results.validation.lqr_force_max_difference_N=max(abs(results.comparison.lqr.u_N(:)-
reference.comparisons(1).lqr.u_N(:)));
assert(results.validation.lqr_angle_max_difference_deg<1e-6);
assert(results.validation.lqr_force_max_difference_N<1e-5);
assert(results.comparison.mpc.metrics.peak_force_N<=10+1e-7);
assert(results.comparison.mpc.metrics.recovered);
fid=fopen(fullfile(folder,'project4_results.json'),'w'); c=onCleanup(@()fclose(fid));
fprintf(fid,'%s\n',jsonencode(results,'PrettyPrint',true));
disp(results.comparison.lqr.metrics); disp(results.comparison.mpc.metrics);
close_system(old,0); close_system(model,0);
end

function result=extract_case(output,label)
t=(0:0.02:30)'; z=sample(output.state_log,t,'linear');
if strcmp(label,'LQR'), method='linear'; else, method='previous'; end
u=sample(output.force_command,t,method);
assert(all(isfinite(z),'all') && all(isfinite(u)));
if strcmp(label,'MPC')
    [rawtime,idx]=unique(double(output.force_command.Time(:)),'last');
    rawforce=double(output.force_command.Data(:)); rawforce=rawforce(idx);
    force_effort=sum(diff(rawtime).*rawforce(1:end-1).^2);
else
    force_effort=trapz(t,u.^2);
end
a=z(:,3)*180/pi; outside=find(abs(a)>1 | abs(z(:,1))>.05,1,'last');
recovered=isempty(find(abs(a)>=90,1)) && max(abs(a(t>=28)))<=1 && max(abs(z(t>=28,1)))<=.05;
settling=-1; if recovered, if isempty(outside), settling=0; elseif outside<numel(t), settling=t(outside+1);
end, end
result=struct('label',label,'time_s',t','theta_deg',a','x_m',z(:,1)', ...
    'u_N',u','metrics',struct('recovered',recovered,'settling_time_s',settling, ...
    'peak_angle_deg',max(abs(a)),'peak_cart_m',max(abs(z(:,1))), ...
    'peak_force_N',max(abs(double(output.force_command.Data(:)))),'effort_N2_s',force_effort, ...
    'saturation_s',trapz(t,double(abs(u)>=10-1e-6))));
end
function y=sample(signal,t,method)
[tx,idx]=unique(double(signal.Time(:)),'stable');
data=reshape(double(signal.Data),numel(signal.Time),[]);
y=interp1(tx,data(idx,:),t,method,'extrap');
end
```

The upright linearization and LQR gain used by the comparison are calculated as follows.

```
function [K, design] = design_lqr_controller(M, m, ell, c_theta, g)
% Design state feedback for z = [x; x_dot; theta; theta_dot].
% Theta is measured in radians from upright; force is in newtons.

A = [0 1 0 0; ...
     0 0 -m*g/M c_theta/(M*ell); ...
     0 0 0 1; ...
     0 0 (M+m)*g/(M*ell) -c_theta*(M+m)/(M*m*ell^2)];
```

```
B = [0; 1/M; 0; -1/(M*ell)];

% Selected penalties for position, velocity, angle, angular velocity.
% The input penalty balances state regulation against control effort.
Q = diag([4 2 300 10]);
R = 0.5;
controllability_rank = rank(ctrb(A, B));
[K, ~, poles] = lqr(A, B, Q, R);

design = struct('A', A, 'B', B, 'Q', Q, 'R', R, 'K', K, ...
    'poles_real', real(poles), 'poles_imag', imag(poles), ...
    'controllability_rank', controllability_rank);
end
```

The validation script checks the saved model and compares the three prediction horizons.

```
function validate_project4()
% Reopen the standalone model and verify the constrained sampled controller.
folder=fileparts(mfilename('fullpath')); cd(folder);
Simulink.fileGenControl('set','CacheFolder',fullfile(tempdir,'emch792-project4-cache'), ...
    'CodeGenFolder',fullfile(tempdir,'emch792-project4-code'),'createDir',true);
resultfile=fullfile(folder,'project4_results.json');
r=jsondecode(fileread(resultfile)); model='inverted_pendulum_mpc';
load_system(model); ws=get_param(model,'ModelWorkspace'); base=getVariable(ws,'mpc_controller');
% Load and run the file without the original project's workspace.
out=sim(model);
[peak,effort,statusmin,statusmax]=raw_checks(out);
assert(peak<=10+1e-7 && statusmin>0);
t=(0:0.02:30)'; z=sample(out.state_log,t,'linear');
r.validation.standalone_angle_max_difference_deg=max(abs(z(:,3)*180/pi-r.comparison.mpc.theta_deg(:)));
assert(r.validation.standalone_angle_max_difference_deg<1e-8);
r.validation.raw_peak_force_N=peak;
r.validation.qp_status_min=statusmin; r.validation.qp_status_max=statusmax;
r.validation.standalone_passed=true;
r.comparison.mpc.metrics.effort_N2_s=effort;
r.comparison.mpc.metrics.peak_force_N=peak;
r.comparison.mpc.metrics.effort_method='Exact integral of piecewise-constant logged force';
% Repeat using shorter prediction horizons and the same controller weights.
ps=[20 35 50];
for i=1:numel(ps)
  if ps(i)==50
    run=out;
  else
    alt=base; alt.PredictionHorizon=ps(i); assignin(ws,'mpc_controller',alt);
    run=sim(model);
  end
  [pk,en,qmin,~]=raw_checks(run); zz=sample(run.state_log,t,'linear'); aa=zz(:,3)*180/pi;
  good=all(isfinite(zz),'all') && max(abs(aa(t>=28)))<=1 && max(abs(zz(t>=28,1)))<=0.05 && max(abs(aa))<90;
  outside=find(abs(aa)>1 | abs(zz(:,1))>0.05,1,'last'); st=-1;
  if good, if isempty(outside), st=0; elseif outside<numel(t), st=t(outside+1); end, end
  r.horizon_study(i)=struct('steps',ps(i),'seconds',ps(i)*0.05,'recovered',good, ...
      'settling_time_s',st,'peak_cart_m',max(abs(zz(:,1))), ...
      'peak_force_N',pk,'effort_N2_s',en,'status_min',qmin);
  assert(pk<=10+1e-7 && qmin>0);
end
assignin(ws,'mpc_controller',base);
% Export the actual Simulink block diagram for the report appendix.
try
    open_system(model); set_param(model,'PaperPositionMode','auto');
    print(['-s' model],'-dpng','-r180',fullfile(folder,'mpc_model_diagram.png'));
catch err
    warning('Diagram export failed: %s',err.message);
end
close_system(model,0);
fid=fopen(resultfile,'w'); cleaner=onCleanup(@() fclose(fid));
fprintf(fid,'%s\n',jsonencode(r,'PrettyPrint',true));
disp(r.validation); disp(r.horizon_study);
end
```

```
function [peak,effort,statusmin,statusmax]=raw_checks(out)
u=out.force_command; peak=max(abs(u.Data(:))); t=double(u.Time(:)); x=double(u.Data(:));
[t,ind]=unique(t,'last'); x=x(ind);
% Applied controller force is piecewise constant between recorded updates.
effort=sum(diff(t).*x(1:end-1).^2);
s=double(out.qp_status.Data(:)); statusmin=min(s); statusmax=max(s);
assert(all(isfinite(s)) && all(s>0));
end
function y=sample(signal,t,method)
[tx,idx]=unique(double(signal.Time(:)),'stable');
data=reshape(double(signal.Data),numel(signal.Time),[]);
y=interp1(tx,data(idx,:),t,method,'extrap');
end
```

The layout functions arrange and export the model and verify the saved configuration.

```
function format_project4_layout(model)
% Arrange the main diagram and route each signal for readability.
blocks={'Base disturbance',[90 245 195 285];'Cart force',[275 250 305 280]; ...
 'Nonlinear plant',[385 205 515 325];'State dimensions',[565 225 595 255]; ...
 'MPC Controller',[660 195 745 315];'Actuator limit',[800 245 850 285]; ...
 'Upright reference',[525 350 615 380];'Command log',[900 245 1015 285]; ...
 'Optimizer status',[800 355 920 385];'Disturbance log',[205 445 315 485]; ...
 'Total force log',[365 445 475 485];'State log',[560 445 670 485]};
lines=find_system(model,'FindAll','on','SearchDepth',1,'Type','line');
for i=1:numel(lines)
    try, delete_line(lines(i)); catch, end
end
for i=1:size(blocks,1), set_param([model '/' blocks{i,1}],'Position',blocks{i,2}); end
set_param([model '/State dimensions'],'ShowName','off');
edges={'Base disturbance/1','Cart force/2';'Base disturbance/1','Disturbance log/1'; ...
 'Cart force/1','Nonlinear plant/1';'Cart force/1','Total force log/1'; ...
 'Nonlinear plant/1','State dimensions/1';'Nonlinear plant/1','State log/1'; ...
 'State dimensions/1','MPC Controller/1';'Upright reference/1','MPC Controller/2'; ...
 'MPC Controller/1','Actuator limit/1';'MPC Controller/2','Optimizer status/1'; ...
 'Actuator limit/1','Cart force/1';'Actuator limit/1','Command log/1'};
for i=1:size(edges,1), add_line(model,edges{i,1},edges{i,2},'autorouting','on'); end
end
```

```
function format_and_verify()
folder=fileparts(mfilename('fullpath')); cd(folder);
Simulink.fileGenControl('set','CacheFolder',fullfile(tempdir,'emch792-project4-cache'), ...
 'CodeGenFolder',fullfile(tempdir,'emch792-project4-code'),'createDir',true);
model='inverted_pendulum_mpc'; load_system(model); format_project4_layout(model);
save_system(model); output=sim(model);
assert(max(abs(output.force_command.Data(:)))<=10+1e-7 && all(output.qp_status.Data(:)>0));
r=jsondecode(fileread('project4_results.json')); t=(0:.02:30)';
[tx,idx]=unique(double(output.state_log.Time(:)),'stable');
z=reshape(double(output.state_log.Data),numel(output.state_log.Time),[]);
z=interp1(tx,z(idx,:),t,'linear');
assert(max(abs(z(:,3)*180/pi-r.comparison.mpc.theta_deg(:)))<1e-8);
open_system(model); set_param(model,'PaperPositionMode','auto');
print(['-s' model],'-dpng','-r180',fullfile(folder,'mpc_model_diagram.png'));
close_system(model,0); fprintf('Reformatted model reopened and verified.\n');
end
```